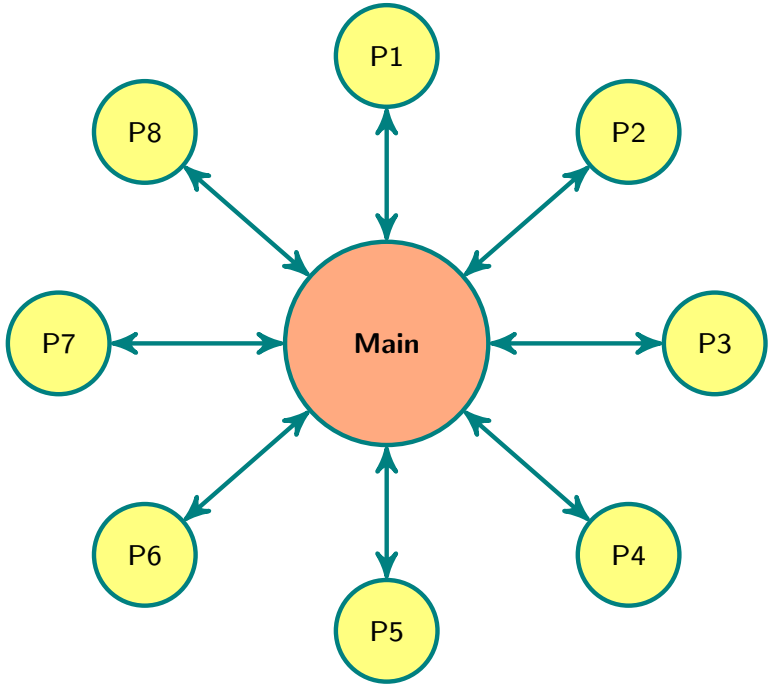
Interprocess Communication of Algebraic Data

Antony Della Vecchia

TU Berlin

ISSAC 2026-07-15



Serializing Algebraic Data

Serialization: translate in-memory data to a form that can be stored or transmitted, then reconstructed on another process or machine.

Say we want to store:

$$p = 2y^3z^4 + (\mathbf{a} + 3)z^2 + 5\mathbf{a}y + 1$$

Serializing Algebraic Data

Serialization: translate in-memory data to a form that can be stored or transmitted, then reconstructed on another process or machine.

Say we want to store:

$$p = 2y^3z^4 + (\mathbf{a} + 3)z^2 + 5\mathbf{a}y + 1$$

- Is y considered a coefficient of z?
- What is $\mathbf{a}$?

Serializing Algebraic Data

Serialization: translate in-memory data to a form that can be stored or transmitted, then reconstructed on another process or machine.

Say we want to store:

$$p = 2y^3z^4 + (\mathbf{a} + 3)z^2 + 5\mathbf{a}y + 1$$

- Is y considered a coefficient of z?
- What is $\mathbf{a}$?
- How can we guarantee the objects behave as expected on load?

Serializing Algebraic Data

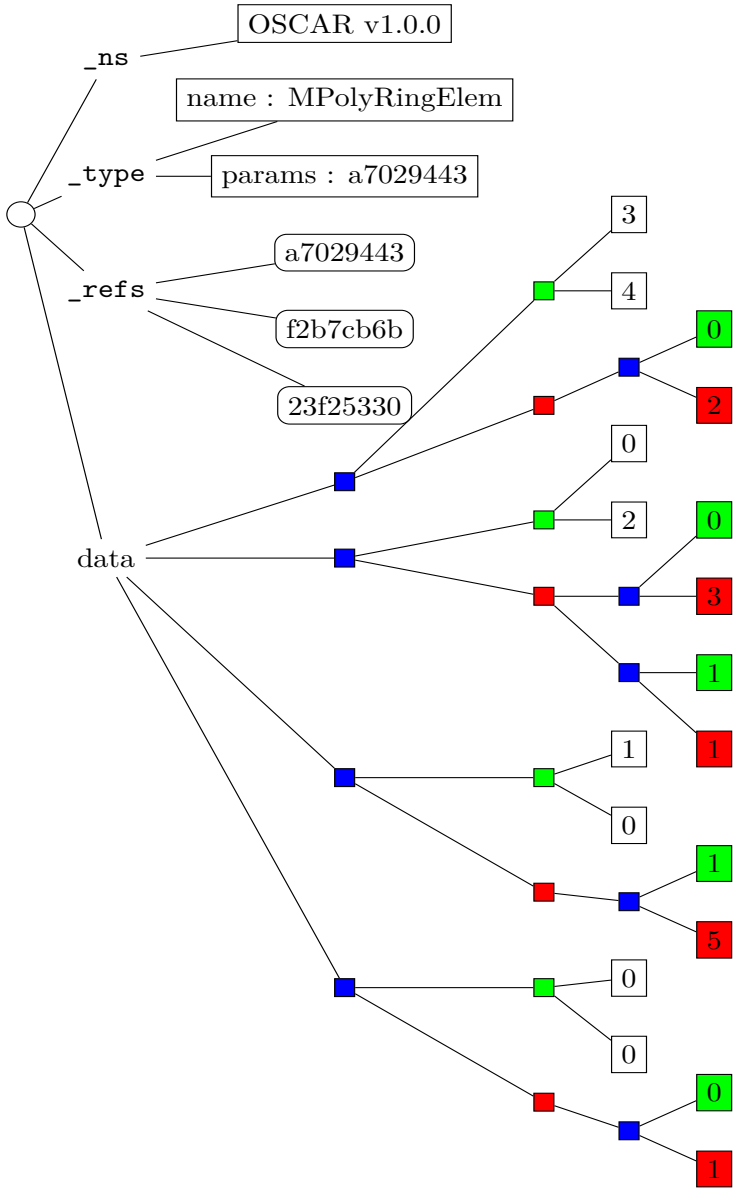
Serialization: translate in-memory data to a form that can be stored or transmitted, then reconstructed on another process or machine.

Say we want to store:

$$p = 2y^3z^4 + (\mathbf{a} + 3)z^2 + 5\mathbf{a}y + 1$$

- Is y considered a coefficient of z?
- What is $\mathbf{a}$?
- How can we guarantee the objects behave as expected on load?
- Elements sharing a parent should have the same parent on load.

The `mrdi` File Format [24 DV - Joswig - Lorenz]



$2y^3z^4$

$(a+3)z^2$

$5ay$

1

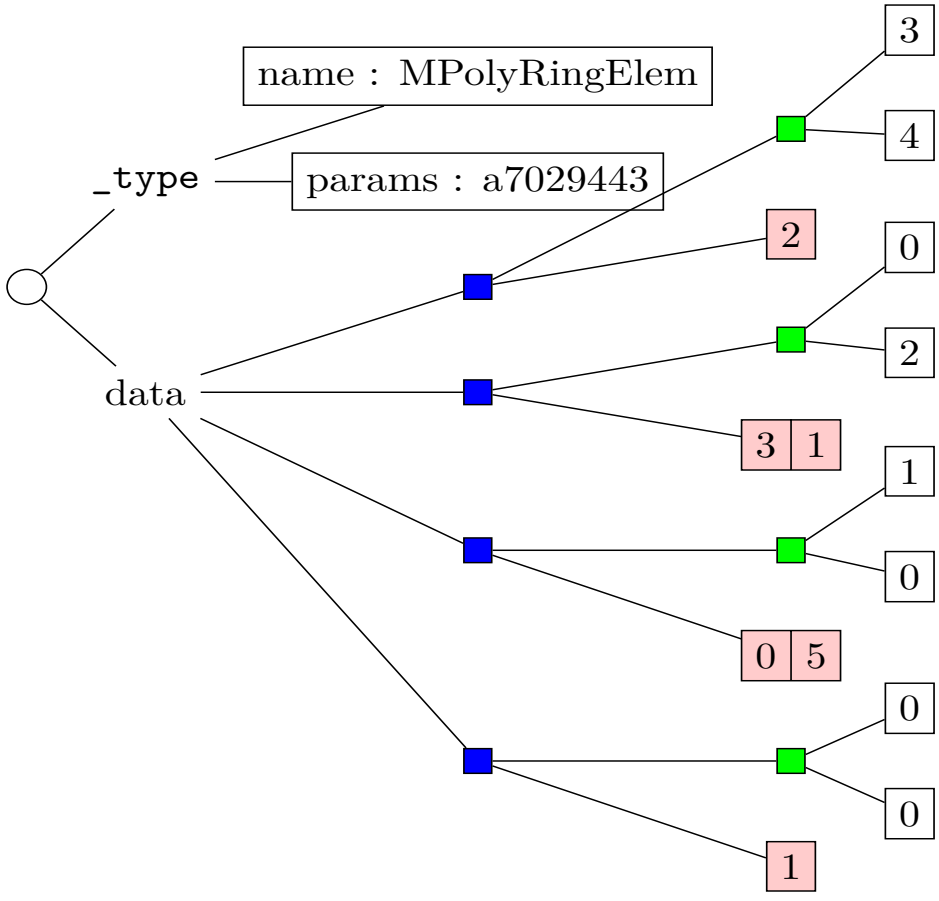
Serializers

- Selection of different serializers allow users to alter subtrees.
- Parametrized `SerializerState{S}`.
- Multiple dispatch can select the encoding per context.

```
function save_object(s::SerializerState, p::PolyRingElem)
  save_object(s, [
    (i - 1, c) for (i, c) in enumerate(coefficients(p))
      if !is_zero(c)
    ]
  )
end

function save_object(s::SerializerState{IPCSerializer}, p::PolyRingElem)
  save_object(s, collect(coefficients(p)))
end
```

Encoding for p during IPC



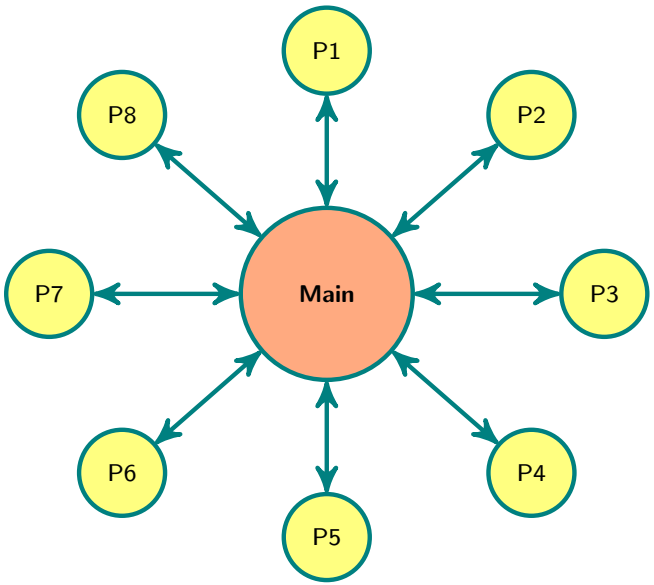
$2y^3z^4$

$(a+3)z^2$

$5ay$

1

`Distributed.jl` Integration



`OscarWorkerPool` — subtype of `Julia`'s `AbstractWorkerPool`.

- Handles sending context to workers.
- `Julia`'s `remotecall` and `pmap` works directly with `OSCAR` types.
- Works with cluster managers (`SlurmClusterManager.jl`) — 1000+ cpus.

Motivating Examples

Finding generators for Vanishing ideals Example: Degree five components of the vanishing ideal for $\sigma_4(\mathbb{P}^3 \times \mathbb{P}^3 \times \mathbb{P}^3)$

- Multigraded Implicitization [26 Cummings-Hollering]
- 8,303,632 monomials over 175,616 multidegrees
- Row reductions run in parallel, one per multidegree
- **32 processes,** 150 GB**:** < 3 **hours vs.** > 24 **hours (1 process)**

Motivating Examples

Finding generators for Vanishing ideals Example: Degree five components of the vanishing ideal for $\sigma_4(\mathbb{P}^3 \times \mathbb{P}^3 \times \mathbb{P}^3)$

- Multigraded Implicitization [26 Cummings-Hollering]
- 8,303,632 monomials over 175,616 multidegrees
- Row reductions run in parallel, one per multidegree
- **32 processes,** 150 GB**:** < 3 **hours vs.** > 24 **hours (1 process)**

Modular determinant, real plane algebraic curves:

- 48×48 matrix over $\mathbb{Z}[x]$
- CRT: reduce modulo enough primes, reconstruct over $\mathbb{Z}[x]$
- **6 processes: 357 s vs. 1326 s (1 process)**

Examples: `https://github.com/dmg-lab/OscarParallelExamples`

Future Work

- Writing to binary formats to reduce communication overhead.
- Make framework available as a standalone `Julia` package.

Future Work

- Writing to binary formats to reduce communication overhead.
- Make framework available as a standalone `Julia` package.

Thank You!